

0.7. La même ligne, avec do ... while

La boucle qui vérifie sa condition à la fin : elle fait toujours au moins un tour.

1. Ce qu'il faut comprendre

`do { ... } while (i < 20);` fait le tour **d'abord**, et vérifie la condition **ensuite**. Il fait donc **toujours au moins un tour**, même si la condition est fausse dès le départ.

Ici, le résultat est le même qu'avec `for` et `while` : les trois boucles savent faire la même chose. On choisit celle qui se lit le mieux.

Attention au **point-virgule** après `while (i < 20)` : il est obligatoire avec `do`, et seulement avec lui.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1330 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

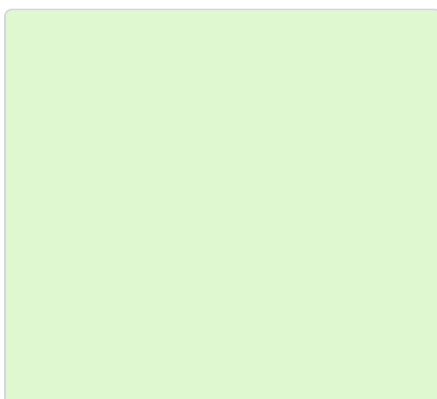
```
int main() {
    uint8_t i = 0;

    do {
        poser(i, 0, ALPHABET[i]); // la lettre n° i, dans la colonne i
        i++;
    } while (i < 20);              // la condition est vérifiée À LA FIN

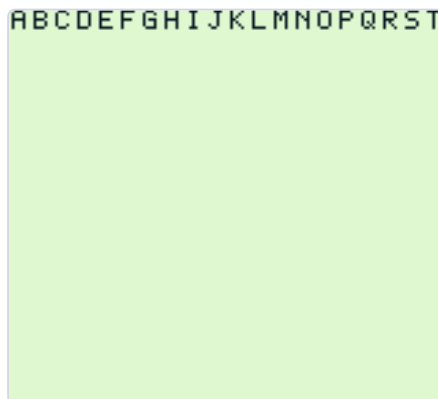
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur toute la première ligne, comme au 0.5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 est pleine : A à T
- ✓ rien ne déborde sur la ligne 1

Ensuite : **0.7.1. La même ligne, avec `do ... while` — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++